Vani — Machine Lane: Spec v0.1 (2026-11-03)

Status: SPEC ONLY — Deepak's design, given in chat Nov 3, 2026. No code written. Build starts on Deepak's GO, phase by phase (§7). Status update (Nov 3, 2026, ~21:40 CT): Phases 1–2 per §7 (registry + trigger + rules engine + alert call + outcome callback) AND Phase 3's v1 inbound (recognition + fork + push snapshots) are BUILT and DEPLOYED on the hosted Vani service, with the Command Center Admin tab (status cards, rules editor, registry manager, test trigger) live on the panel. All offline suites pass (service Section T + panel admin suite); NO live call has been placed yet — the lane is PENDING Deepak's trial (register a test customer, fire the test trigger from the Admin tab). Terminology: "the IoT platform" in this document always means **the Things Alive IoT Platform (Platform 2.0)** — never a generic or third-party platform. Sibling document: `ThingsAlive_ThingsAI_VoiceAgent_Blueprint_20261028.md` (the SDR lane blueprint; this spec extends the same agent, it does not replace it).

1. Vision — one Vani, two lanes

Vani is one voice with two jobs, and both jobs run on the same pattern: **a context packet goes in, a grounded conversation comes out.** What changes between the lanes is whose packet she carries.

	SDR lane (exists)	Machine lane (this spec)
Audience	Prospects (sales & marketing)	Registered customers of the IoT platform
Context packet	**Contact packet** — dossier research, hook, picture, prior outcomes	**Machine packet** — one specific machine's alert, health card, prediction, readings
Outbound trigger	Deepak arms a call from the Command Center	The IoT Platform fires an alert / trigger action (§3)
Inbound	Prospect calls back; recognized by number	Registered customer calls; recognized by number (§4)
Goal of the call	Book a 20-minute meeting with Deepak	Tell the customer what happened on their machine, what it means, what to do next

Deepak's design, in his words (Nov 3, 2026): *"for machine lane i will register the customer and his contact number; the customer will get a call when machines send alerts or based on the trigger actions from iot platform, so need an end point for iot platform to trigger the outgoing call; then the same*

registered client when he calls, vani identifies the client by the incoming number and will ask for which machine the enquiry is, or to review over machines updates under his account."

2. Customer registry

Deepak registers each customer manually in v1. The registered phone number is the **identity key in both directions**: it is where alert calls are dialed, and it is how an inbound caller is recognized.

2.1 Fields (per account)

- `account_id` — stable id, assigned at registration; the platform uses it in trigger requests (§3).
- `account_name` — the customer / company name, as Vani should say it.
- `contact_name` — the person Vani greets and addresses.
- `registered_numbers[]` — one or more E.164 numbers, each flagged `primary` or `secondary`. These are the only numbers that can receive this account's alert calls or unlock this account's data inbound.
- `machines[]` — per machine: `machine_id` (the platform's id), `name` (as spoken), `type`, `site`.
- `timezone` — the customer's local time; drives quiet-hours policy (§3.5).
- `alert_preferences` — the account's policy settings once Deepak sets them (severity floor for calls, quiet hours, caps — §3.5 decision slots).
- `callback_url` (optional) — the platform endpoint that receives call outcomes for this account (§3.4); a single platform-wide default may be configured instead.
- `status` — `active` / `alert_calls_off` (opt-out state, §3.5).

2.2 Where it lives

A **separate store** in the Vani service (proposed: `data/customers.json`, hand-maintained like the existing `data/manual_contacts.json`). It must NOT be folded into `data/contacts.json`: that index is auto-rebuilt by `vani/contacts.py` from campaign call sheets and call history, and a rebuild would erase hand-registered customers.

2.3 Recognition

Inbound recognition reuses the **built** caller-ID recognition (`vani/contacts.py`, keyed by E.164, tolerant of common US number formats). The registry is checked as part of that lookup: a match on a registered number resolves the caller to their account and their machines before Vani speaks her first line.

3. Outbound alert calls — the trigger endpoint contract

This section is the contract the platform team builds against. Their side can be completed from this section alone.

The platform does not place calls and does not speak. It POSTs one trigger per alert occurrence to the Vani service; Vani decides (per policy, §3.5) whether the trigger earns a call, places it, and reports the outcome back (§3.4).

3.1 Endpoint

```
POST {VANI_BASE_URL}/platform/alert-call
Authorization: Bearer <PLATFORM_TOKEN>
Content-Type: application/json
```

- The endpoint is a **sibling of the existing token-gated** `POST /calls` and follows the same discipline: no valid credential, no service (401).
- It uses its **own credential** (`PLATFORM_TOKEN`), distinct from the Command Center's `VANI_API_TOKEN`, so either can be rotated or revoked without touching the other. The token is issued to the platform team by Deepak; it never appears in this document or in chat.
- The hosted base URL is the Railway deployment (`https://vani-service-production.up.railway.app` as of Nov 3, 2026; confirm the current base with Deepak before wiring — the path and body below are the stable contract).

3.2 Request body

```
{
  "event_id": "plt-evt-9f27c1",
  "account_id": "cust-acme-plant-2",
  "customer_phone": "+1XXXXXXXXXX",
  "machine": {
    "machine_id": "mch-4471",
    "name": "Air Compressor 2",
    "type": "rotary screw compressor",
    "site": "Plant 2, Bay B"
  },
  "trigger": {
    "kind": "alert",
    "severity": "critical",
    "title": "Bearing failure predicted",
    "fired_at": "2026-11-03T14:05:00-06:00"
  },
  "context": {
    "health_card_summary": "Health score trending down for 6 days; currently in the watch band.",
    "prediction": {
      "statement": "Inner-race bearing failure likely within 10-14 days at current load.",
      "confidence": "high"
    },
    "signals": [
```

```
      { "name": "Vibration (overall)", "value": "8.2", "unit": "mm/s", "threshold": "7.1" },
      { "name": "Motor current", "value": "112", "unit": "A", "threshold": "105" }
    ],
    "recommended_action": "Plan a bearing swap at the next scheduled stop; avoid continuous full-
load running until then.",
    "notes": "Second occurrence this month; the first self-cleared on Oct 19."
  },
  "callback_url": "https://platform.example/outcomes/vani",
  "language": "en-US"
}
```

Field rules:

- `event_id` (**required**) — the platform's unique id for this alert occurrence. **Idempotency key**: a repeat POST with the same `event_id` for the same account returns the original acceptance and places **no** second call.
- `account_id` (**required**) — must match the registry. `customer_phone` is an optional cross-check; if both are present and disagree, the request is rejected (422), never guessed.
- `machine` (**required**) — `machine_id` + spoken `name`; `type` and `site` help Vani place the machine for the customer.
- `trigger.kind` — `alert` (a machine sent an alert) or `trigger_action` (a platform trigger action fired). `trigger.severity` — the platform's severity for this event (see the mapping note in §8). `trigger.fired_at` — ISO 8601, with timezone.
- `context` (**required**) — **the machine packet**, composed by the platform and pushed with the trigger, exactly as SDR packets carry dossier research. Vani speaks only from this packet (§5). Every field the platform wants said aloud must be in here, in words a person can hear — values with units, the prediction in a sentence, the confidence as the platform states it.
- `callback_url` (optional) — overrides the account's configured outcome callback (§3.4).
- `language` (optional) — defaults to `en-US`.

3.3 Responses

- `202 Accepted` — `{ "call_id": "...", "event_id": "...", "status": "queued" }`. The call is placed asynchronously; acceptance means Vani owns the trigger now.
- `200` with `"status": "duplicate"` and the original `call_id` — the `event_id` was already accepted.
- `401` — missing/invalid platform credential.
- `422` — validation failed (unknown `account_id`, unregistered phone, malformed packet), with an `error` field saying which. Nothing is dialed on a 422.
- A trigger can be **accepted but not dialed** when policy says it doesn't earn a call (§3.5). That is not an error: the outcome callback (§3.4) reports `held_for_inbound_review` or `suppressed_by_policy`, so the platform is never left guessing.

3.4 Outcome callback

When the call (or the policy decision) settles, Vani POSTs to the account's `callback_url`:

```
{
  "event_id": "plt-evt-9f27c1",
  "call_id": "...",
  "account_id": "cust-acme-plant-2",
  "machine_id": "mch-4471",
  "outcome": "completed_spoke_with_customer",
  "summary": "Customer acknowledged; will schedule the bearing swap at Friday's stop and wants a
follow-up after.",
  "duration_s": 96,
  "attempt": 1,
  "called_at": "2026-11-03T14:07:31-06:00"
}
```

- `outcome` values: `completed_spoke_with_customer` · `voicemail_dropped` · `voicemail_not_delivered` · `no_answer` · `held_for_inbound_review` · `suppressed_by_policy` · `failed` (with the reason in `summary`).
- `summary` — one line, written for the platform's alert history, in the same plain-words discipline as the call itself.
- Vani retries a failed callback POST a bounded number of times; the platform should dedupe on `event_id` (a retried callback is the same outcome, not a new one).

3.5 Call flow (what the customer hears)

1. **Disclosure + identity, first breath** — Vani names herself as the AI voice of Things Alive (same standing rule as every lane; she never speaks as Deepak), names the account/machine context: she is calling about *their* machine.
2. **The machine and the alert in plain words** — machine name and site, what fired, when.
3. **What it means** — from the packet's prediction and health-card summary, with the confidence stated as the platform stated it.
4. **The recommended action** — the packet's `recommended_action`, verbatim in substance.
5. **The offer** — connect to a human now (the built escalation bridge), or schedule a follow-up; she books only what the customer agrees to.
6. **Close** — one-line recap of what the customer decided; the outcome + summary are written and callback-fired (§3.4).

Unanswered: the call rolls to the **deterministic voicemail drop** (proven Nov 3, 2026 in Deepak's own mailbox — on a machine verdict the live call redirects and Twilio's own voice speaks the armed message; no model speaks into the mailbox). The alert voicemail text is composed from the same packet: machine, alert, recommended action, and the callback number (+1 817-809-3797).

3.6 Policy — DEEPAK DECISIONS (slots, with recommended defaults)

These are policy, not engineering. The build takes whatever Deepak sets here; the defaults below are recommendations only, **not** his answers.

- **[DECISION — severity floor]** Which severities earn a phone call, and which wait for the inbound review (§4)? *Recommended default:* `critical` calls immediately; `warning` calls only inside the customer's business hours, else held for review; `info` never calls.
- **[DECISION — retries]** How many attempts on an unanswered alert call, and how spaced? *Recommended default:* 2 attempts, 30–60 minutes apart, the deterministic voicemail drop on the final attempt; then stop and report `no_answer`.
- **[DECISION — quiet hours]** Hours during which alert calls are not placed, in the customer's local time. *Recommended default:* 8:00 PM – 8:00 AM held for `warning` and below. Whether `critical` is exempt from quiet hours is part of this decision — flagged both ways, not assumed.
- **[DECISION — storm caps]** Per-machine and per-account daily call caps, so an alert storm cannot machine-gun a customer. *Recommended default:* 3 calls/machine/day, 6 calls/account/day; past the cap, triggers are accepted, logged, and reported as `suppressed_by_policy` for the inbound review.
- **[DECISION — opt-out]** What a spoken "stop calling me about alerts" does. *Recommended default:* alert calls off for that account (`status: alert_calls_off`) until Deepak re-enables them; the platform's own notification channels are the platform's business and are untouched.

4. Inbound machine enquiries

The same registered client calls the 817 number. Flow:

1. **Recognition** — the incoming number resolves to the account (§2.3) before the first line. A registered caller is greeted **by name**, with their account context loaded.
2. **The fork question** (Deepak's design): *"Which machine is the enquiry about — or shall I review the latest updates under your account?"*
3. **Named machine** → she answers from that machine's data: health-card state, latest predictions with confidence, recent alerts and what became of them — only what the data supply (§4.1) holds, nothing invented (§5).
4. **Account review** → she walks the account's machines in priority order (open alerts first, then predictions, then anything changed since the last review), one machine at a time, letting the customer stop or dive in.
5. **Unregistered number** → the existing generic inbound mode (identify, learn the need, route) — unchanged. An unregistered caller is never told anything about any account's machines (§5).

The seed of this flow already exists in the live inbound experience-center flow — the machine branch opens: *"Hi, welcome to Things Alive — voice to your Machines. Please let me know what is the machine you want to talk to."* The machine lane replaces that generic branch with the recognized-account fork above.

4.1 Data supply for the inbound direction — two options, one recommendation

Outbound is push by design (the packet rides the trigger, §3.2). Inbound has no trigger, so the account's machine data must reach Vani another way:

- **Option A — Push snapshots (v1 recommendation).** The platform POSTs an account/machine snapshot to Vani's store whenever a machine's state materially changes (same credential family as §3; a snapshot endpoint sibling to the trigger endpoint). The inbound conversation reads the latest snapshot. *Why v1:* it needs no new read API on the platform, trials can start with hand-pushed snapshots, and the store doubles as the record of what Vani knew. *Cost:* a snapshot can be stale between changes — so every snapshot carries a `generated_at`, and Vani speaks freshness honestly ("as of this morning's update") instead of implying live data.
- **Option B — Pull at call time (target).** Vani queries a platform read API when the call starts (account → machines → current state). Always current, but the platform must build and secure a read surface before the first inbound trial can happen.

Spec both; **build A first, migrate to B when the platform read API exists.** The conversation layer must not care which supplied the packet — same machine-packet schema as §3.2's `context`, extended with per-machine `recent_alerts[]` and `last_reviewed_at` for the account-review walk.

5. Grounding + safety (standing rules, carried into this lane unchanged)

- **Packet-only speech.** Vani speaks only from the machine packet / snapshot in front of her, in the IoT Platform's 2.0 capability vocabulary — health cards, live predictions with confidence, the machine-knowledge libraries (Configuration, Analytics, Prediction, Diagnostic, Health Cards). If a value isn't in the packet, she says she doesn't have it. She never invents a reading, a threshold, a date, or a confidence.
- **Data isolation.** An account's machines, alerts, and history are reachable **only** from that account's registered number(s). A caller is never told another account's data — and never told whether another account or machine exists.
- **Disclosure + identity.** AI disclosure in the first breath of every call, both directions. Vani speaks as Things Alive's voice, never as Deepak; escalation to a human uses the built bridge.
- **No internal defects, ever.** Callers never hear about platform internals, faults, or fixes. The maturity standing line applies verbatim where maturity comes up: *"The platform is a work in*

progress — we're landing patch updates as we go. What I can show you today is what the features do."

- **Opt-out is instant.** A spoken opt-out on any call takes effect on that call (§3.6 slot) and is reported in the outcome callback.

6. What exists vs what's new (honest table)

Piece	Status
Caller recognition by incoming number (`vani/contacts.py`)	**EXISTS** — built, live on inbound
Token-gated `POST /calls` (Bearer token, 401 otherwise)	**EXISTS** — Nov 3, 2026; hosted deployment enforces it
Deterministic voicemail drop (Twilio `<Say>` redirect)	**EXISTS** — proven Nov 3, 2026 in Deepak's own mailbox ("Voicemail dropped")
Inbound machine-branch welcome ("voice to your Machines...")	**EXISTS** — in the live inbound flow (§4 seed)
Escalation bridge to a human (`escalate_to_deepak`)	**EXISTS** — built, live
Hosted Vani service on Railway, `/health`, per-call transcripts + outcome logging	**EXISTS** — live since Nov 3, 2026
Customer registry (`data/customers.json`) + a way for Deepak to register/edit accounts	**NEW**
`POST /platform/alert-call` + the platform credential	**NEW**
Machine-packet schema in the session layer (alert-call prompt composition)	**NEW**
Alert-voicemail text composer (from the packet)	**NEW**
Severity / retry / quiet-hours / caps policy config (§3.6)	**NEW** — values are Deepak's decisions
Outcome callback to the platform (§3.4)	**NEW**
Inbound account-review data supply (snapshot endpoint v1; pull API target)	**NEW**

Piece	Status
Registry + alert-call history surface in the Command Center	**NEW** — rides the already-directed Command Center move to Railway; scheduled with that build, not this spec

7. Build order (each phase closes on a trial, not on tests)

Standing completion standard (Deepak, Oct 30, 2026): nothing is "completed" on schema or unit tests — a phase is done when Deepak has **experienced it on a trial call to his own number** (+1 201-203-1340), registered for this purpose as the first test customer with a test machine.

1. **Phase 1 — Registry + outbound trigger.** Registry store, `POST /platform/alert-call` + credential, machine-packet composition, policy config with Deepak's §3.6 values. *Trial:* a platform-shaped test trigger fires an alert call to Deepak; a second trigger is let to ring → deterministic voicemail drop; the outcome callback lands on a test listener. Duplicate `event_id` re-POST places no second call.
2. **Phase 2 — Inbound account review.** Registry recognition on inbound, the fork question, snapshot endpoint + pushed test snapshots. *Trial:* Deepak calls in from his registered number, is greeted by name, names the test machine, then takes the account review.
3. **Phase 3 — Platform integration.** The platform team wires real triggers and the real callback URL; Deepak registers the first real customer; the first real alert call is made under his review of its transcript.
4. **Phase 4 — Target upgrades.** Pull-based live reads (§4.1 Option B) when the platform read API exists; Command Center surface for the registry and alert history, with the panel's Railway build.

8. Open questions — flagged, not filled

These were ambiguous or unstated in the design as given; the spec does not silently decide them.

1. **Multiple registered numbers — ring order.** The registry supports several numbers per account (§2.1), but the design says "his contact number" (singular). On an alert, does Vani ring the primary only, or fall to the secondary if the primary doesn't answer? Needs Deepak's rule before Phase 1 policy is final.
2. **A number that is both an SDR contact and a registered customer.** Recognition will match both stores. Proposed default — the customer identity wins inbound and the machine fork is offered — but this is a proposal, not Deepak's word.
3. **Severity vocabulary mapping.** §3.2 assumes the platform can send `critical | warning | info`. If the platform's native severity scale differs, the platform team and Deepak need a mapping table before §3.6's severity floor can be enforced.

4. **Who is "the human" on a machine-lane escalation?** The built bridge dials Deepak. A customer-support rota may be the right target for machine alerts; the bridge target for this lane is Deepak's decision.
5. **Callback addressing.** §3 allows a per-account `callback_url` or one platform-wide default. Which the platform prefers is theirs to say at integration time (Phase 3); the contract supports both.